Failover Mechanisms

detect, decide, promote, redirect

Watch the 4-min video

In one line: failover detects that the primary is gone, confirms it, promotes a standby, fences the old primary and redirects clients; its downtime is the sum of those steps, and its data loss is whatever replication hadn't copied yet.

1 Noon, and the primary is gone

Your primary database dies at noon. A standby is ready in another zone. Two numbers matter now: **how long** until writes work again (your RTO, lesson 13), and **how much data** was lost (your RPO). Failover design is about both.

2 Four steps

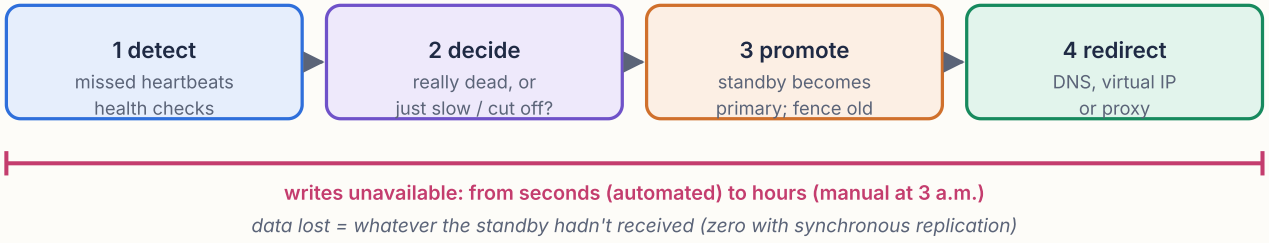


Figure 1. Failover time is detection + decision + promotion + redirection. Every step can be the slow one.

Automated failover is fast, often under a minute, but can act on a false alarm and **flap** back and forth. **Manual** failover lets a human confirm but takes minutes to hours. Many teams automate zone-level failover of a database and keep a human in the loop for region-level failover (lesson 15).

3 Detection and split brain

Detection is a trade-off: a short timeout (say 3 missed heartbeats at 1 s) reacts fast but fires on network blips; a long one is safe but extends the outage. The worst outcome is **split brain**: the old primary was only cut off, not dead, and keeps accepting writes while the new one does too.

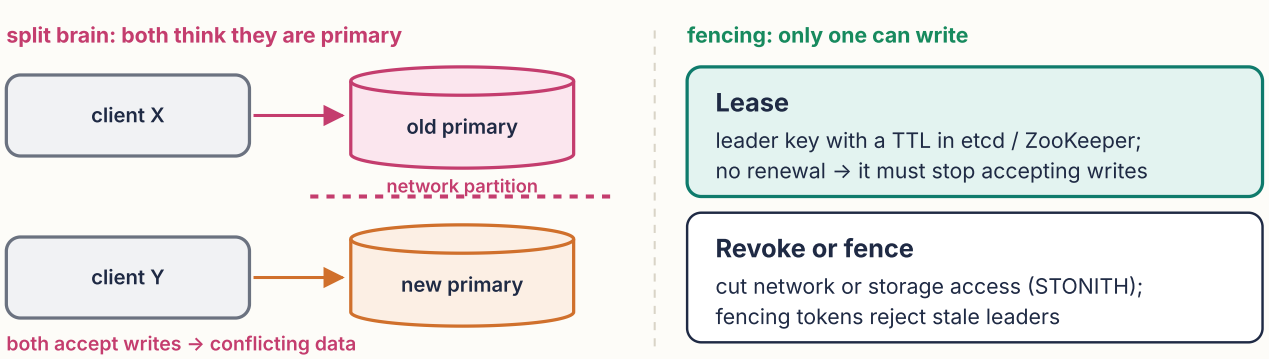


Figure 2. Fencing makes sure the old primary can't write after the new one takes over.

Tools build this in: **Patroni** for PostgreSQL keeps the leader key in etcd or Consul with a TTL; **Redis Sentinel** needs a quorum of sentinels to agree before promoting a replica; managed services such as

Amazon RDS Multi-AZ handle detection, promotion and fencing for you.

4 Redirecting clients, and what is lost

Redirection	How it works	Watch out for
DNS update	the endpoint name is repointed to the new primary	cached records: keep TTL short (e.g. 5–30 s), clients must re-resolve
Virtual IP	a floating IP moves to the new primary	usually limited to one network segment / zone
Proxy	HAProxy, PgBouncer, RDS Proxy always routes to the leader	the proxy itself needs redundancy
Client discovery	clients ask Sentinel / etcd who leads	every client library must support it

With **synchronous** replication every committed write is already on the standby, so nothing committed is lost. With **asynchronous** replication the standby lags; writes inside that lag window are lost. Either way, **in-flight transactions abort**, so clients must reconnect and retry with backoff (lesson 5).

WORKED EXAMPLE: MANAGED POSTGRES, MULTI-AZ

A managed PostgreSQL instance runs in multi-AZ mode with a **synchronous standby** in zone B. Zone A fails. The service detects it, promotes the standby and repoints the database's DNS name, which AWS documents as typically 60–120 seconds for RDS Multi-AZ. No committed data is lost. Open connections drop, so the app's connection pool reconnects with retries; users see a short burst of errors, then recovery. The team tests this regularly with a forced failover (reboot with failover).

COMMON MISTAKES

- No fencing, so a partitioned old primary keeps taking writes.
- Long DNS TTLs, or clients that resolve once at startup and never again.
- Apps that crash on a dropped connection instead of reconnecting with backoff.
- Never testing failover, then discovering a missing step during a real outage.

5 Key terms

Failover: moving the primary role to a standby

Split brain: two nodes both acting as primary

Lease: time-limited leadership that must be renewed

Promotion: making a replica the new writable primary

Fencing: preventing the old primary from writing

Replication lag: how far an async standby is behind

6 Interview questions

Q1. What are the steps of failover? BASIC

Detect the failure, decide it's real, promote the standby (and fence the old primary), then redirect clients to it.

Q2. Automated or manual failover? BASIC

Automated is fast but can act on false alarms and flap; manual is safer but slow. Automate the common case, keep a human for big decisions like region failover.

Q3. How long does failover take, and what can be lost? SCENARIO

Detection + decision + promotion + redirection: seconds to a few minutes for managed databases. Synchronous replication loses no committed writes; asynchronous loses the replication lag. In-flight transactions abort and must be retried.

Q4. After failover, half your app servers still write to the old primary. Why? SCENARIO

They cached the old DNS answer or hold old connections; use short TTLs, re-resolve on reconnect, and fence the old primary so its writes fail.

Q5. What's the biggest failover risk? TRICKY

Split brain: the old primary keeps accepting writes. Prevent it with fencing or leases.

Q6. Why not set the detection timeout to one second? TRICKY

Brief network blips or GC pauses would trigger failovers that cause more downtime and risk flapping than the blip itself.

Next: SD-K-012 · Disaster Recovery backendroadmap.com